

Lab10 – WebClient

1. Reactive Programming vs Block - ต่างกันอย่างไร

ตอบ Blocking คือ thread ต้องรอจนงาน (เช่น query DB, เรียก API) เสร็จก่อนถึงจะทำงานถัดไปได้ ทำให้ต้องใช้ thread จำนวนมากเมื่อมี request พร้อมกันเยอะๆ ส่วน Reactive Programming คือ รูปแบบการเขียนโปรแกรมแบบ non-blocking/asynchronous โดย thread ไม่ต้องรอผลลัพธ์ แต่จะ "subscribe" เพื่อรับผลเมื่อข้อมูลพร้อม ทำให้ใช้ thread น้อยลงแต่รองรับ concurrent request ได้มากขึ้น เหมาะกับงานที่ต้องรอ I/O เยอะ เช่น เรียก API หรือ DB

2. Mono vs Flux — ใช้กรณีไหน และทำไม

ตอบ ทั้งคู่เป็น Publisher ใน Reactor library

- **Mono<T>** ใช้แทนข้อมูล 0 หรือ 1 ค่า เช่น หา Product ตาม id (findById), บันทึกข้อมูล (save), หรือการลบที่ไม่คืนค่า (Mono<Void>)
- **Flux<T>** ใช้แทนข้อมูลหลายค่า (0 ถึง N) เช่น ดึง Product ทั้งหมด (findAll) หรือกรองตาม category (findByCategory)

3. WebClient — อธิบาย method chain .get(), .uri(), .retrieve(), .bodyToMono/Flux()

ตอบ WebClient คือ HTTP client แบบ reactive สำหรับเรียก API ภายนอก โดย method chain มีความหมายดังนี้

- **.get()** — กำหนด HTTP method ที่จะใช้ (GET/POST/PUT/DELETE)
- **.uri("...")** — กำหนด endpoint/URL ปลายทางที่จะเรียก
- **.retrieve()** — ส่ง request จริงและเตรียมรับ response กลับมา
- **.bodyToMono(Class) / .bodyToFlux(Class)** — แปลง response body ให้เป็น Mono หรือ Flux ของ type ที่ต้องการ (map JSON → Object)

4. Code ของ Repository, Service, Controller พร้อม comment อธิบาย

- Repository (ProductRepository — เก็บข้อมูลจำลองด้วย ConcurrentHashMap ทำหน้าที่ CRUD พื้นฐาน คื้นค่าเป็น Mono/Flux ตามชนิดข้อมูล

```
package com.example.lab10.repository;

import com.example.lab10.model.Product;
import reactor.core.publisher.Flux;
import reactor.core.publisher.Mono;

import java.util.Map;
import java.util.concurrent.ConcurrentHashMap;

/**
 * ProductRepository — In-memory Reactive Repository
 *
 *  โครงสร้างและ annotation ครบแล้ว
 *  TODO: เติม method body ให้ครบทุก method
 *
 * ใช้ ConcurrentHashMap เป็น in-memory storage
 * (ไม่ต่อ Database — เน้นฝึก Mono/Flux)
 *
 * Hint:
 * - Mono.just(value) คื้นค่าเดียว
 * - Mono.empty() คื้นเปล่า
 * - Flux.fromIterable(list) คื้นหลายค่าจาก collection
 */
public class ProductRepository {
```

// — In-memory storage

```
private final Map<String, Product> store = new ConcurrentHashMap<>();
```

// — Constructor: ใส่ข้อมูลตัวอย่าง

```
public ProductRepository() {
    store.put("1", new Product("1", "iPhone 15 Pro (673380425-2 SEC 3)",
        "Electronics", "Apple", 50, 39900.0, "MEMBER"));
    store.put("2", new Product("2", "MacBook Air M3",
        "Electronics", "Apple", 20, 49900.0, "NONE"));
    store.put("3", new Product("3", "Samsung Galaxy S24",
        "Electronics", "Samsung", 30, 29900.0, "SEASONAL"));
}
```

// — 1. หา Product 1 รายการ

```
/**
 * TODO: คีน Mono<Product> จาก store โดยใช้ id
 * ถ้าไม่พบให้คีน Mono.empty()
 *
 * Hint: store.get(id) คีน Product หรือ null
 * ถ้า null ให้ใช้ Mono.empty()
 * ถ้ามีค่าให้ใช้ Mono.just(product)
 */
public Mono<Product> findById(String id) {
    Product product = store.get(id);

    if (product != null) {
        return Mono.just(product);
    }
}
```

```
    } else {
        return Mono.empty();
    }
}
```

// — 2. หา Product ทั้งหมด

```
/**
 * TODO: คืน Flux<Product> ของทุกรายการใน store
 *
 * Hint: store.values() คืน Collection<Product>
 * ใช้ Flux.fromIterable(...) แปลงเป็น Flux
 */
public Flux<Product> findAll() {
    return Flux.fromIterable(store.values());
}
```

// — 3. บันทึก Product

```
/**
 * TODO: บันทึก product ลง store แล้วคืน Mono<Product>
 *
 * Hint: store.put(product.getId(), product)
 * แล้วใช้ Mono.just(product) คืนค่า
 */
public Mono<Product> save(Product product) {
    if (product == null) {
        return Mono.empty();
    }
    store.put(product.getId(), product);
}
```

```

        return Mono.just(product);
    }

    // — 4. ลบ Product



---


    /**
     * TODO: ลบ product จาก store แล้วคืน Mono<Void>
     *
     * Hint: store.remove(id)
     * แล้วใช้ Mono.empty() คืนค่า (Mono<Void>)
     */
    public Mono<Void> deleteById(String id) {
        store.remove(id);
        return Mono.empty();
    }

    // — 5. กรองตาม category



---


    /**
     * TODO: คืน Flux<Product> ที่ category ตรงกัน
     *
     * Hint: findAll()
     * .filter(p -> p.getCategory().equalsIgnoreCase(category))
     */
    public Flux<Product> findByCategory(String category) {
        return findAll()
            .filter(p -> p.getCategory().equalsIgnoreCase(category));
    }
}

```

- Service (ProductService) — ชั้น business logic เรียกใช้ Repository, เพิ่ม logic เช่น generate UUID ตอน save, throw error ตอนหาไม่เจอด้วย switchIfEmpty, และคำนวณราคา หลังส่วนลดด้วย .map()

```
package com.example.lab10.service;

import com.example.lab10.model.Product;
import com.example.lab10.repository.ProductRepository;

import java.util.UUID;

import org.springframework.stereotype.Service;
import reactor.core.publisher.Flux;
import reactor.core.publisher.Mono;

/**
 * ProductService — Business Logic Layer
 *
 *  @Service, Constructor Injection ครบแล้ว (DIP — SOLID)
 *  TODO: เติม method body ให้ครบทุก method
 *
 * * หน้าที่: รับ request จาก Controller → เรียก Repository → คืนผล
 * * (SRP — แต่ละ class มีหน้าที่เดียว)
 *
 * * Hint Operators ที่ควรใช้:
 * * .map(p -> ...) แปลงค่า
 * * .flatMap(p -> ...) async transform
 * * .defaultIfEmpty(...) fallback ถ้าว่าง
 * * .switchIfEmpty(Mono...) fallback Mono ถ้าว่าง
 */
```

```

@Service
public class ProductService {

    // — Constructor Injection (DIP — SOLID)
    

---


    private final ProductRepository repository;

    public ProductService(ProductRepository repository) {
        this.repository = repository;
    }

    // — 1. ดึง Product 1 รายการ
    

---


    /**
     * TODO: เรียก repository.findById(id) แล้วคืนผล
     * ถ้าไม่พบให้ throw RuntimeException("Product not found: " + id)
     *
     * Hint: repository.findById(id)
     * .switchIfEmpty(Mono.error(new RuntimeException(...)))
     */
    public Mono<Product> getById(String id) {
        return repository.findById(id)
            .switchIfEmpty(Mono.error(new RuntimeException("Product not
found: " + id)));
    }

    // — 2. ดึง Product ทั้งหมด
    

---


    /**
     * TODO: เรียก repository.findAll() แล้วคืนผล

```

```
*/
public Flux<Product> getAll() {
    return repository.findAll();
}
```

// — 3. บันทึก Product

```
/**
 * TODO: เรียก repository.save(product) แล้วคืนผล
 *
 * เพิ่มเติม: ถ้า product.getId() เป็น null ให้ generate id ใหม่
 * Hint: java.util.UUID.randomUUID().toString()
 */
public Mono<Product> save(Product product) {
    if (product.getId() == null || product.getId().isBlank()) {
        product.setId(UUID.randomUUID().toString());
    }
    return repository.save(product);
}
```

// — 4. ลบ Product

```
/**
 * TODO: เรียก repository.deleteById(id) แล้วคืนผล
 */
public Mono<Void> delete(String id) {
    return repository.deleteById(id);
}
```

```
// — 5. กรองตาม category

/**
 * TODO: เรียก repository.findByCategory(category) แล้วคืนผล
 */
public Flux<Product> getByCategory(String category) {
    return repository.findByCategory(category);
}

// — 6. คำนวณราคาหลังส่วนลด

/**
 * TODO: หา Product จาก id แล้วคืน discountedPrice
 *
 * Hint: getById(id)
 * .map(p -> p.getDiscountedPrice())
 */
public Mono<Double> getDiscountedPrice(String id) {
    return repository.findById(id).map(p -> p.getDiscountedPrice());
}
}
```

- Controller (ProductController) — รับ HTTP request ผ่าน @RestController แล้วส่งต่อให้ Service ทำงาน คืนผลเป็น Mono/Flux กลับไปเป็น JSON response โดยอัตโนมัติ (WebFlux จัดการ subscribe ให้)

```
package com.example.lab10.controller;

import com.example.lab10.model.Product;
import com.example.lab10.service.ProductService;
```

```
import org.springframework.web.bind.annotation.*;
import reactor.core.publisher.Flux;
import reactor.core.publisher.Mono;

/**
 * ProductController — Reactive REST Controller
 *
 *  @RestController, @RequestMapping, Constructor Injection ครบแล้ว
 *  endpoint GET /products/{id} ทำเสร็จแล้วเป็นตัวอย่าง (30%)
 *  TODO: เติม method body ของ endpoint ที่เหลือ (70%)
 *
 * Endpoints ที่ต้องทำทั้งหมด:
 * GET /products → Flux<Product> (ดึงทั้งหมด)
 * GET /products/{id} → Mono<Product>  ตัวอย่างทำแล้ว
 * POST /products → Mono<Product> (บันทึก)
 * DELETE /products/{id} → Mono<Void> (ลบ)
 * GET /products/category/{cat} → Flux<Product> (กรอง)
 * GET /products/{id}/price → Mono<Double> (ราคาหลังลด)
 */
@RestController
@RequestMapping("/products")
public class ProductController {

    // — Constructor Injection (DIP — SOLID)

    private final ProductService service;

    public ProductController(ProductService service) {
        this.service = service;
    }
}
```

```
}
```

```
//
```

```
//  ตัวอย่างที่ทำเสร็จแล้ว — ศึกษาแล้วทำ endpoint ที่เหลือ
```

```
//
```

```
/**
```

```
 * GET /products/{id}
```

```
 * คืน Mono<Product> — ค้นหา Product 1 รายการ
```

```
 *
```

```
 * ทดสอบ: GET http://localhost:8080/products/1
```

```
 */
```

```
@GetMapping("/{id}")
```

```
public Mono<Product> getById(@PathVariable String id) {
```

```
    return service.getById(id);
```

```
}
```

```
//
```

```
//  TODO: เติม method body ด้านล่างนี้
```

```
//
```

```
/**
```

```
 * GET /products
```

```
* TODO: คืน Flux<Product> ทุกรายการ
*

* Hint: เรียก service.getAll()
* ทดสอบ: GET http://localhost:8080/products
*/

@GetMapping
public Flux<Product> getAll() {
    return service.getAll();
}

/**
 * POST /products
 * TODO: รับ Product จาก request body แล้วบันทึก
 *
 * Hint: เรียก service.save(product)
 * ทดสอบ: POST http://localhost:8080/products
 * Body: { "name": "...", "price": 999.0, ... }
 */

@PostMapping
public Mono<Product> save(@RequestBody Product product) {
    return service.save(product);
}

/**
 * DELETE /products/{id}
 * TODO: ลบ Product และคืน Mono<Void>
 *
 * Hint: เรียก service.delete(id)
 * ทดสอบ: DELETE http://localhost:8080/products/1
 */
```

```

        @DeleteMapping("/{id}")
        public Mono<Void> delete(@PathVariable String id) {
            return service.delete(id);
        }

        /**
         * GET /products/category/{category}
         * TODO: คืน Flux<Product> ที่กรองตาม category
         *
         * Hint: เรียก service.getByCategory(category)
         * ทดสอบ: GET http://localhost:8080/products/category/Electronics
         */
        @GetMapping("/{category}/{category}")
        public Flux<Product> getByCategory(@PathVariable String category) {
            return service.getByCategory(category);
        }

        /**
         * GET /products/{id}/price
         * TODO: คืน Mono<Double> ราคาหลังส่วนลด
         *
         * Hint: เรียก service.getDiscountedPrice(id)
         * ทดสอบ: GET http://localhost:8080/products/1/price
         */
        @GetMapping("/{id}/price")
        public Mono<Double> getDiscountedPrice(@PathVariable String id) {
            return service.getDiscountedPrice(id);
        }
    }

```